

Helm · Harbor · Argo CD · Zarf

Enterprise Kubernetes application delivery in connected, disconnected and air-gapped environments



Helm

Package



Harbor

Store



Zarf

Transport



Argo CD

Reconcile

Helm packages → Harbor stores → Zarf transports → Argo CD reconciles → Kubernetes runs

Agenda

01

The Problem

Why one tool is never enough

02

Helm

Packaging Kubernetes applications

03

Harbor

Storing and securing artifacts

04

Argo CD

GitOps and continuous reconciliation

05

Zarf

Air-gap-native software delivery

06

Comparison & Decisions

Capability matrix, myths, decision guide

07

Working Together

Connected vs air-gapped reference architectures

08

Labs & Troubleshooting

Python + Redis end-to-end, capstone, debugging

Where this fits in the K3s program



Day 13

Helm & GitOps

Helm architecture, charts, values, releases, rollback · GitOps concepts, Argo CD, FluxCD, continuous delivery



Day 14

Production Operations

HA, multi-node design, load balancing, gateway, distributed storage · deployment and upgrade testing



Day 15

Enterprise & Air-Gapped

Registries, Hauler, Zarf, air-gapped deployments, multi-cluster concepts · final capstone

This module connects Day 13 (packaging & GitOps) to Day 15 (air-gapped enterprise delivery)

01

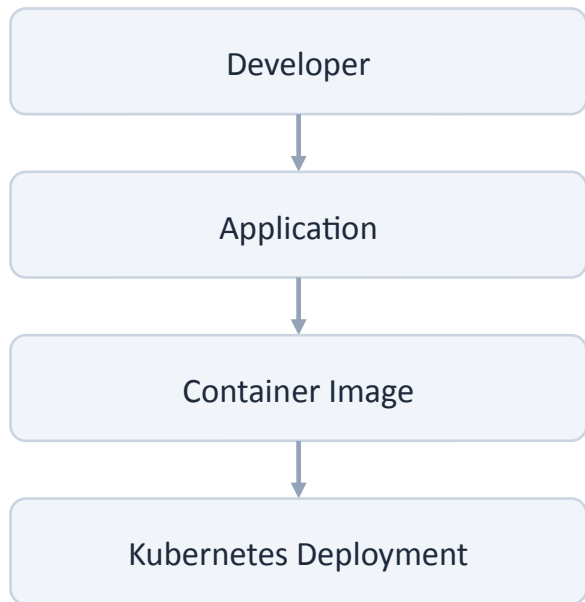
The Problem

Why do we need four different tools to deliver one application?

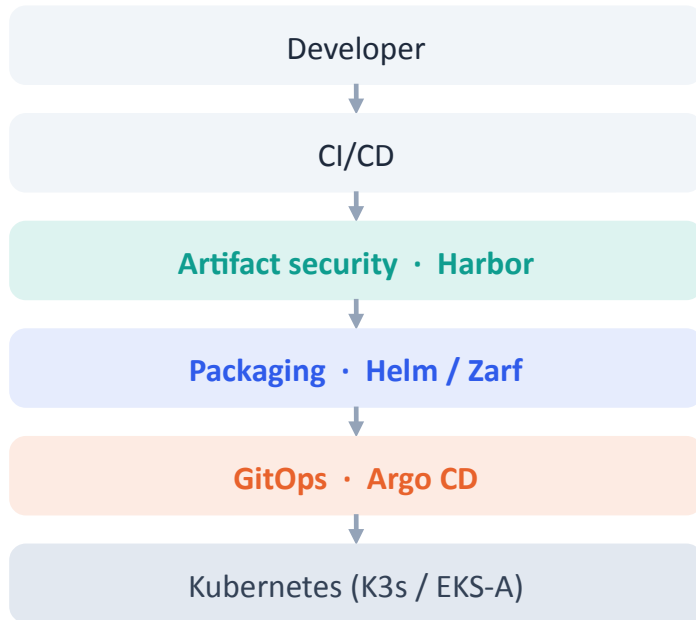


From "kubectl apply" to enterprise delivery

What beginners picture



What enterprises actually run



Every extra stage answers a question the simple model ignores: Is the image trusted? Is it versioned? Does the cluster still match what we approved?

Why do we need four different tools?

"If Helm can deploy an application, why do we need Harbor, Argo CD or Zarf?"



Helm

What are we deploying?

Packages Kubernetes resources into versioned, configurable charts



Harbor

Where are artifacts stored?

Hosts, scans and replicates images, charts and OCI artifacts



Argo CD

How is desired state maintained?

Continuously reconciles the cluster against Git



Zarf

How do we cross the air gap?

Bundles an app and all its dependencies for offline delivery

Four questions, four layers

How do I package Kubernetes resources?

Helm

Where do I store container images and artifacts?

Harbor

How do I continuously deploy from Git?

Argo CD

How do I move an app and its dependencies into an air-gapped environment?

Zarf

They operate at different layers — they complement each other, they don't replace each other

02

Helm

The Kubernetes package manager — "Package my Kubernetes application."



What is Helm?

```
helm create myapp
myapp/
├── Chart.yaml           # metadata
├── values.yaml          # default configuration
├── templates/           # templated manifests
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   ├── ingress.yaml
│   ├── secret.yaml
│   └── _helpers.tpl
└── charts/              # dependencies
```

A chart is a versioned package of templated Kubernetes resources.

Packaging

Versioning

Installation

Upgrade

Rollback

Configuration

Templates + values → rendered manifests → a named, revisioned release in the cluster.

Life without Helm

```
one environment
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f configmap.yaml
kubectl apply -f ingress.yaml
kubectl apply -f secret.yaml
```

× three environments, each with different values

Development

Staging

Production



Copy-pasted YAML

Five files duplicated per environment; small edits diverge quickly



No versioning of the whole app

Nothing ties "these five files" together as release 1.5.0



No clean rollback

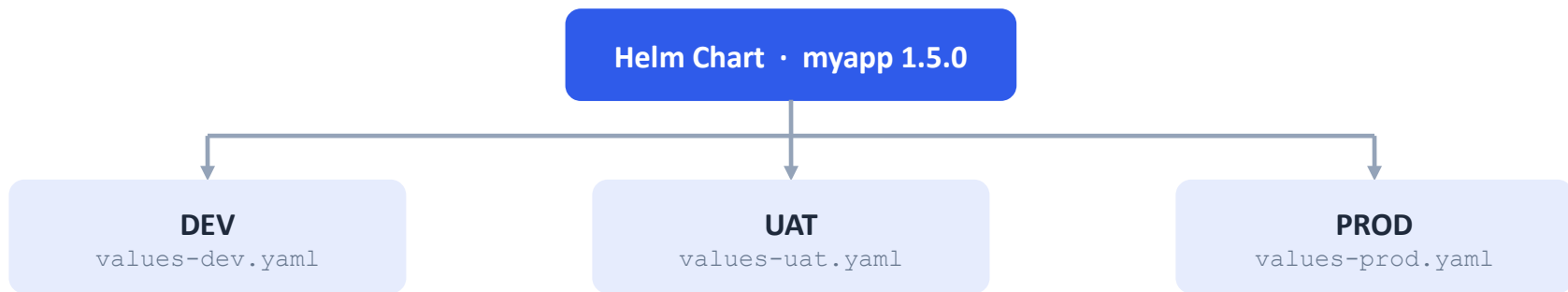
Undoing a bad change means re-applying old files by hand



Ordering & cleanup by hand

Removed resources linger; install order is tribal knowledge

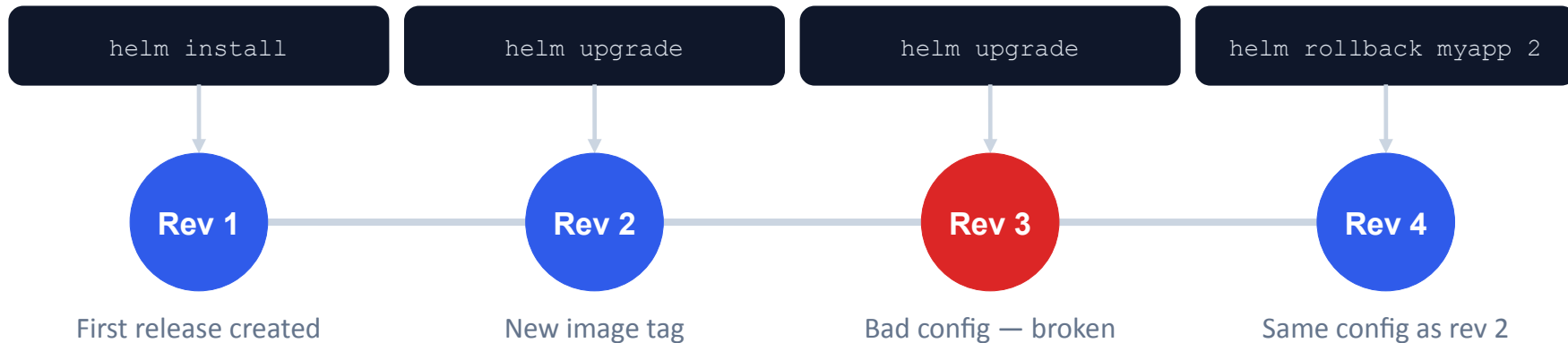
One chart, many environments



```
values-dev.yaml
replicaCount: 1
image:
  repository: harbor.example.com/dev/myapp
  tag: "1.5.0"
ingress:
  enabled: false
```

```
values-prod.yaml
replicaCount: 5
image:
  repository: harbor.example.com/prod/myapp
  tag: "1.5.0"
ingress:
  enabled: true
  host: myapp.example.com
```

Helm release lifecycle



Rollback never rewrites history: it creates a NEW revision (4) that reuses revision 2's configuration.

Check it with: `helm history myapp`

Helm concepts at a glance

Chart

The package: templates + default values + metadata

Repository

Where charts are published — HTTP index or an OCI registry such as Harbor

Release

A named instance of a chart installed in a namespace

Values

Configuration injected into templates (values.yaml, -f, --set)

Templates

Go-templated manifests rendered at install/upgrade time

Revision

Numbered snapshot of a release after each change

Upgrade

Applies a new chart version or new values to a release

Rollback

Redeploys an earlier revision's configuration as a new revision

Helm 3.8+ stores charts in OCI registries: `helm push myapp-1.5.0.tgz oci://harbor.example.com/prod`

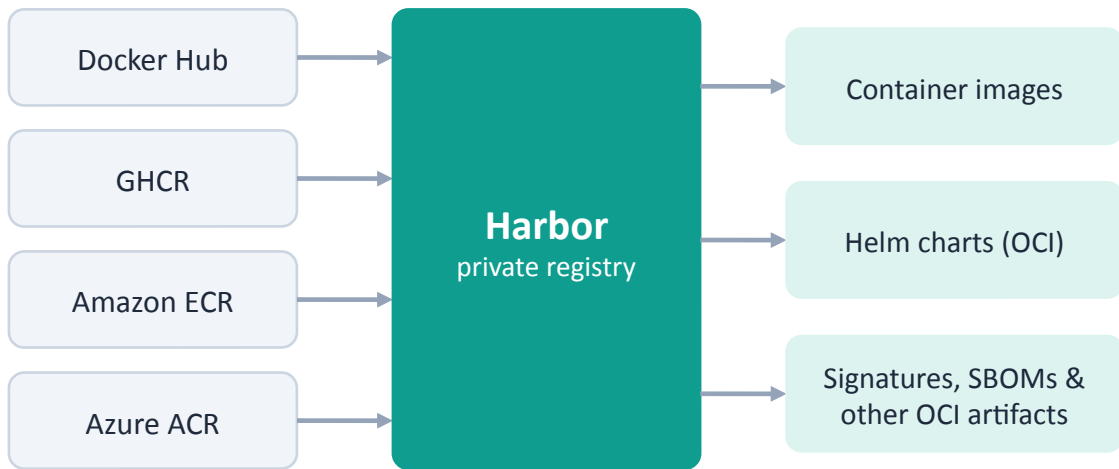
03

Harbor

The enterprise artifact registry — "Store and secure my artifacts."



What is Harbor?

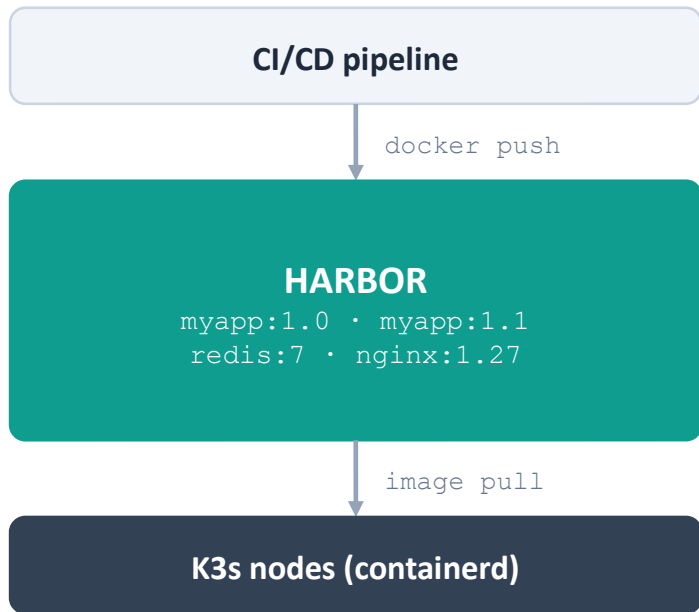


Key capabilities

- Projects with RBAC & robot accounts
- Proxy cache for upstream registries
- Push / pull replication
- Vulnerability scanning (Trivy)
- Signing (Cosign / Notation)
- Quotas, retention & immutability
- Audit logs, OIDC / LDAP

Harbor answers: "Where do my container images and OCI artifacts live?"

Harbor in the Kubernetes environment

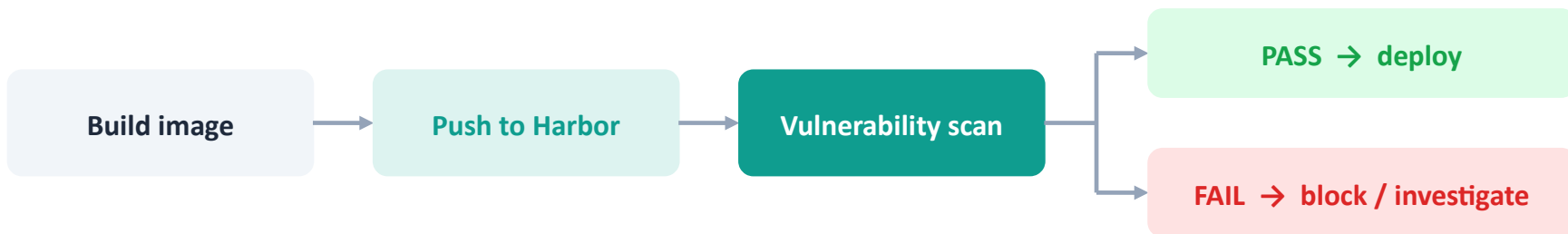


Nodes pull from your private registry, not straight from Docker Hub.

```
/etc/rancher/k3s/registries.yaml
mirrors:
  docker.io:
    endpoint:
      - "https://harbor.example.com"
    rewrite:
      "^(.*)$": "dockerhub-proxy/$1"
configs:
  "harbor.example.com":
    auth:
      username: robot$k3s-puller
      password: <robot-token>
    tls:
      ca_file: /etc/ssl/certs/harbor-ca.crt

# restart k3s / k3s-agent to apply
```

Harbor security: scan before you ship



Pluggable scanners

Trivy is the default scanner. Scan on push, on schedule, or on demand; results are stored per artifact.



Deployment security

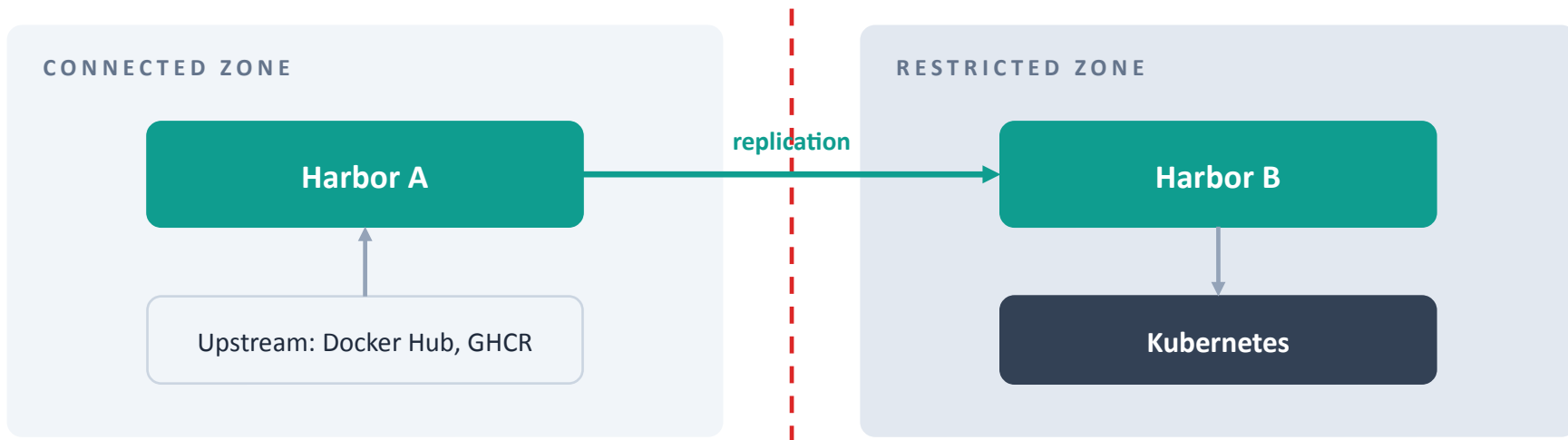
A project can prevent images at or above a chosen severity from being pulled — the gate is enforced at pull time.



Signing & trust

Cosign / Notation signatures are stored as OCI artifacts; a project can allow only signed images to be pulled.

Harbor replication



How it works Push- or pull-based rules; triggered manually, on schedule or on event; filtered by repository, tag or label.

Truly air-gapped? Replication needs a network path. With none, artifacts must be exported to media and imported — the job Zarf (or Hauler) does.

04

Argo CD

GitOps continuous delivery — "Keep Kubernetes synchronized with Git."



What is Argo CD?



Declarative

An Application custom resource declares source (repo, path, revision) and destination (cluster, namespace).

Git is the source of truth

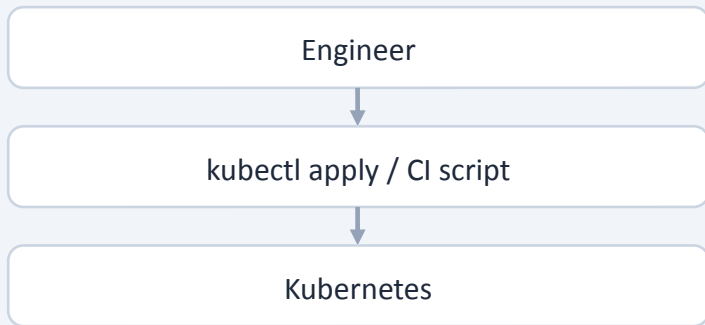
Changes land through commits and pull requests; the cluster follows Git, not people.

Many source formats

Plain YAML, Helm charts, Kustomize and Jsonnet — from Git or from Helm / OCI repositories.

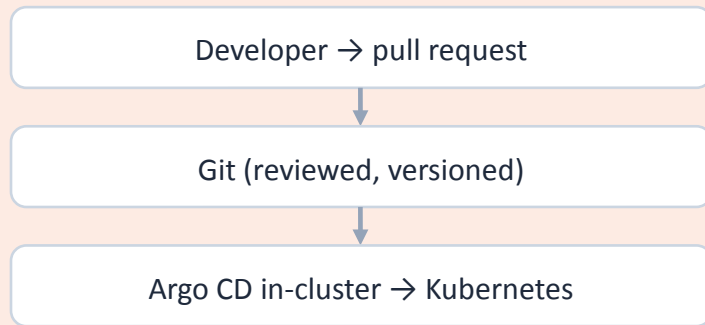
Push vs pull: the GitOps model

Traditional (push)



- ✗ Cluster credentials live on laptops and CI
- ✗ Manual changes are invisible
- ✗ Drift goes unnoticed

GitOps (pull)



- ✓ Every change is reviewed and auditable
- ✓ Cluster pulls; no external credentials
- ✓ Rollback = git revert

Argo CD detects and corrects drift



Automatic correction needs `syncPolicy.automated.selfHeal: true`. Without it, Argo CD flags the app OutOfSync and waits for a manual sync — detection is always on, correction is a choice.

The Argo CD Application resource

```
application.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: python-redis
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://git.example.com/platform/python-redis.git
    targetRevision: main
    path: chart
    helm:
      valueFiles:
        - values-prod.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: demo
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

source

Which repo, branch and path — here a Helm chart plus a values file.

destination

Which cluster and namespace to deploy into.

syncPolicy

Automated sync; prune deletes removed resources; selfHeal reverts drift.

05

Zarf

Air-gap-native software delivery — "Package and carry everything across the gap."



What is Zarf?

An airgap-native package manager for Kubernetes: it bundles everything an application needs into one declarative, portable package.



Air-gapped

No network path to the outside at all — defence, government, critical infrastructure



Disconnected

Intermittently connected sites such as ships, remote mines and field deployments



Constrained

Low-bandwidth or tightly firewalled networks where pulling images is impractical



Standalone / edge

Single-node or edge clusters (K3s) that must be self-sufficient

Bonus: Zarf generates an SBOM for every image it packages — useful for security review before anything crosses the gap.

The air-gap problem

Internet · registries · Git hosts



AIR GAP

Kubernetes

K3s / EKS Anywhere

The application still needs...

Container images

Helm charts

Git repositories

CRDs

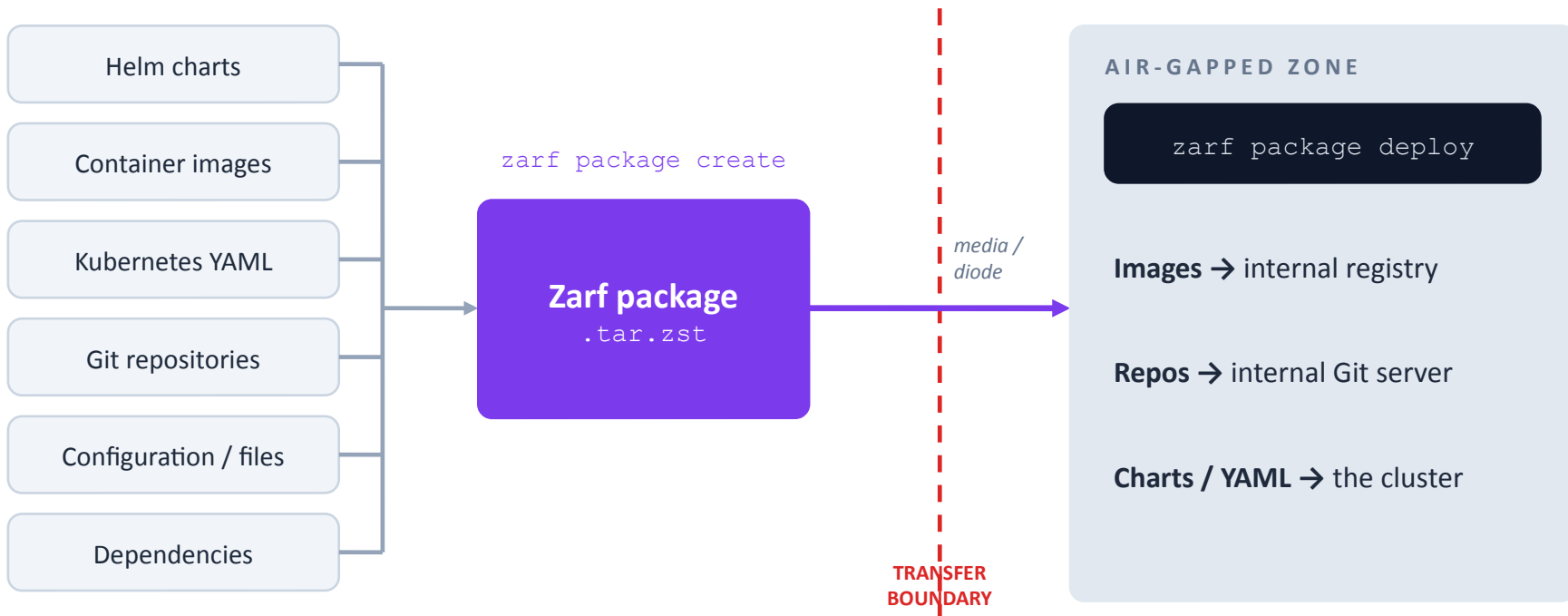
Manifests

Configuration

Dependencies & binaries

How do we get all of this inside — reliably, repeatably and verifiably?

Zarf packaging concept



Inside a Zarf package

conceptual layout

```
zarf-package-python-redis-amd64-1.0.0.tar.zst
├── zarf.yaml           # package definition
├── checksums.txt       # integrity of every file
├── sboms.tar           # SBOM per image
├── images/             # OCI image layout
│   ├── python-api:1.0
│   └── redis:7.2
├── components/
│   └── python-redis.tar
│       ├── charts/     # Helm charts
│       ├── manifests/  # raw YAML
│       ├── repos/      # Git repositories
│       ├── values/     # values files
│       └── files/       # binaries, configs
```

The application and its delivery dependencies travel together.

One file

A single artifact to scan, approve and transfer

Architecture-specific

Built for amd64 or arm64 — must match the target

Verifiable

Checksums, and optional signing with a key

Declarative

Rebuilt identically from the same zarf.yaml

zarf.yaml and the Zarf workflow

```
zarf.yaml
kind: ZarfPackageConfig
metadata:
  name: python-redis
  version: 1.0.0
  description: Python API + Redis demo
components:
- name: python-redis
  required: true
  charts:
    - name: python-redis
      version: 0.1.0
      namespace: demo
      localPath: ./chart
      valuesFiles:
        - values-airgap.yaml
  images:
    - harbor.example.com/training/python-api:1.0
    - docker.io/library/redis:7.2
```

1

CONNECTED**zarf package create .**

Pulls every chart and image, writes the package

2

TRANSFER**scan · approve · carry**

Move the package (and the init package) across

3

AIR-GAPPED**zarf init**

Once per cluster: registry, agent, optional Git server

4

AIR-GAPPED**zarf package deploy <pkg>**

Pushes images, installs charts — fully offline

06

Comparison & Decisions

Which delivery problem does each tool solve?



Helm vs Harbor vs Argo CD vs Zarf

Capability	Helm	Harbor	Argo CD	Zarf
Kubernetes package management	Charts	—	—	Packages
Container image registry	—	Core	—	Bundles images
Helm chart handling	Native	Stores as OCI	Renders & syncs	Bundles & installs
GitOps / continuous reconciliation	—	—	Core	Git mirror only
Drift detection	—	—	—	—
Vulnerability scanning	—	Trivy	—	SBOM generation
Artifact replication	—	Push / pull	—	Offline transfer
Air-gapped delivery	Manual effort	Registry inside gap	With internal sources	Core use case
Rollback	Revisions	—	Git history	Redeploy prior pkg

Don't ask "which one replaces the others?" — ask "which delivery problem does each tool solve?"

Who does what?



Helm

"Package my Kubernetes application."



Harbor

"Store and secure my artifacts."



Argo CD

"Keep Kubernetes synchronized with Git."



Zarf

"Package and transport my complete application into an environment where connectivity is restricted."

Which tool do I use?

Package Kubernetes manifests	Helm	Adopt GitOps	Argo CD
Parameterize deployments per env	Helm	Continuous reconciliation	Argo CD
Store container images	Harbor	Detect configuration drift	Argo CD
Scan images for vulnerabilities	Harbor + scanner	Deploy into a disconnected site	Zarf
Run a private registry	Harbor	Bundle images + charts + manifests	Zarf
Replicate artifacts between sites	Harbor	Bootstrap offline app delivery	Zarf

Myth vs reality

MYTH Zarf is a replacement for Helm.

Zarf uses Helm (its Go SDK) to install charts and even wraps raw manifests as Helm releases.

MYTH Harbor deploys applications.

Harbor stores, scans, signs and replicates artifacts. It never deploys anything.

MYTH Argo CD is a container registry.

Argo CD consumes definitions from Git or chart repos and manages desired state; images still come from a registry.

MYTH Helm is GitOps.

Helm packages and releases apps. GitOps needs a reconciler such as Argo CD or Flux watching Git.

MYTH With Zarf we don't need Harbor.

Zarf carries images in and seeds a registry; Harbor can be the long-lived enterprise registry inside the gap.

07

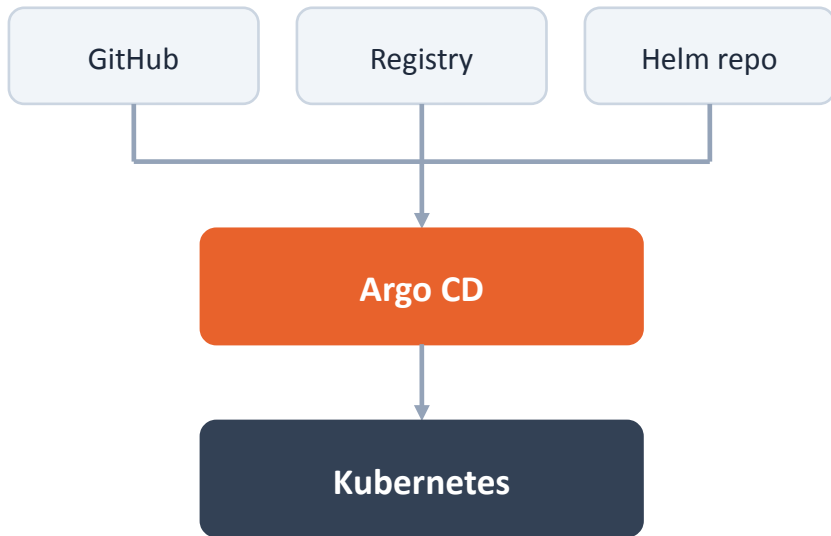
Working Together

From an Internet-connected pipeline to a fully air-gapped reference architecture

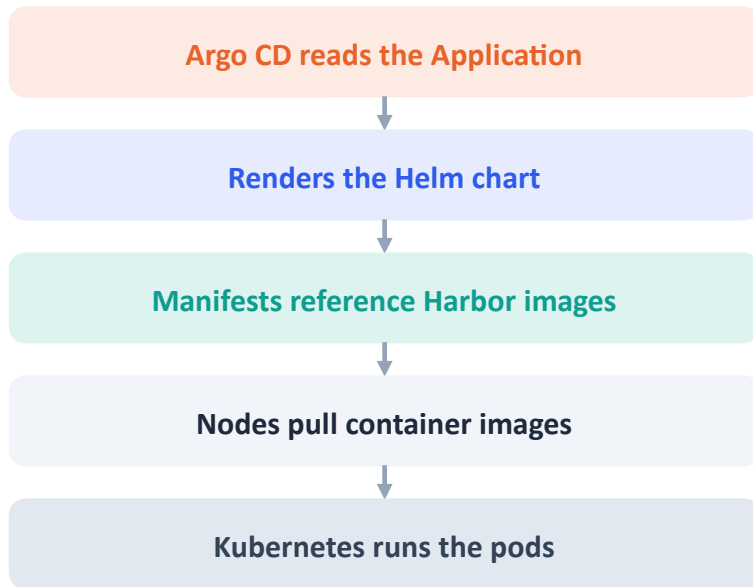


Connected deployment

Everything is reachable over the network



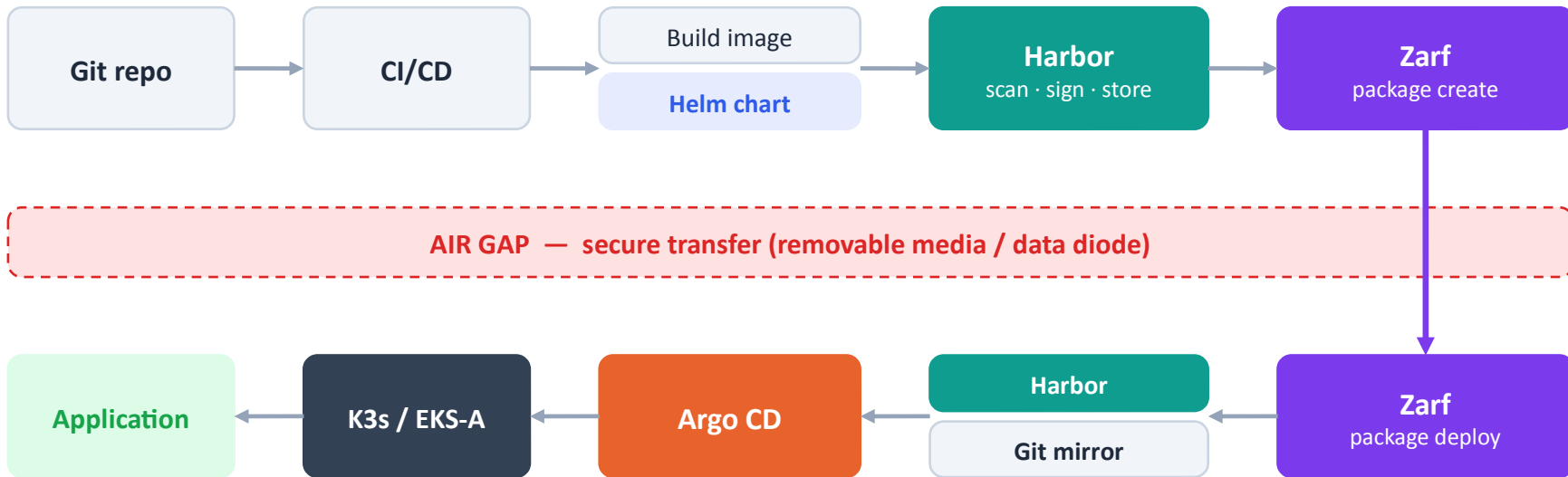
What happens at sync time



No packaging step is needed — until the network disappears.

The complete enterprise pipeline

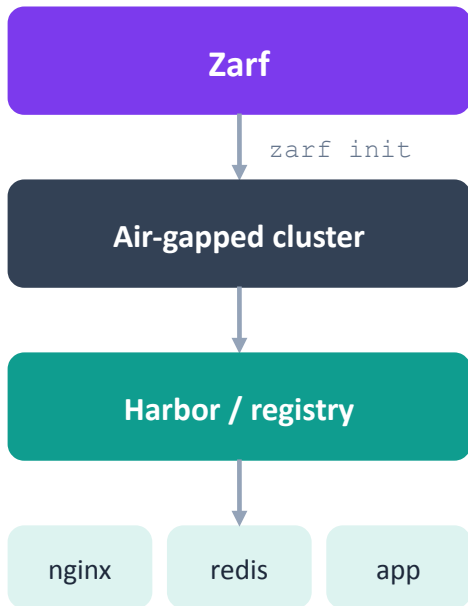
CONNECTED ZONE



AIR-GAPPED ZONE

Read the bottom row right to left: the package lands, feeds the internal registry and Git mirror, and Argo CD takes over.

Zarf + Harbor



Option A — Zarf-managed registry

zarf init deploys an in-cluster registry, the Zarf agent and, optionally, a Gitea Git server. Simple and self-contained.

Option B — Bring your own Harbor

```
zarf init --registry-url=harbor.internal \
  --registry-push-username=<robot> \
  --registry-push-password=<token>
```

Either way

The Zarf agent (a mutating webhook) rewrites image references so pods pull from the internal registry.

Zarf = delivery mechanism · Harbor = artifact repository

Zarf + Helm

Zarf package

Helm chart

deployment.yaml

service.yaml

configmap.yaml

+ images · repos · files



Zarf embeds Helm

Zarf uses Helm's Go SDK to install or upgrade releases — it is a Helm client, not a Helm replacement.



Releases stay visible

helm list -A shows Zarf-deployed releases; history and rollback semantics still apply.



Manifests become charts too

Raw YAML in a component is wrapped into a generated Helm release by Zarf.

Zarf doesn't make Helm obsolete — it uses Helm.

Zarf + Argo CD



Zarf

Gets software INTO the environment

- Initial delivery and bootstrap
- Runs per release, at the gap
- Moves images, charts and repos across
- Seeds the registry and Git server



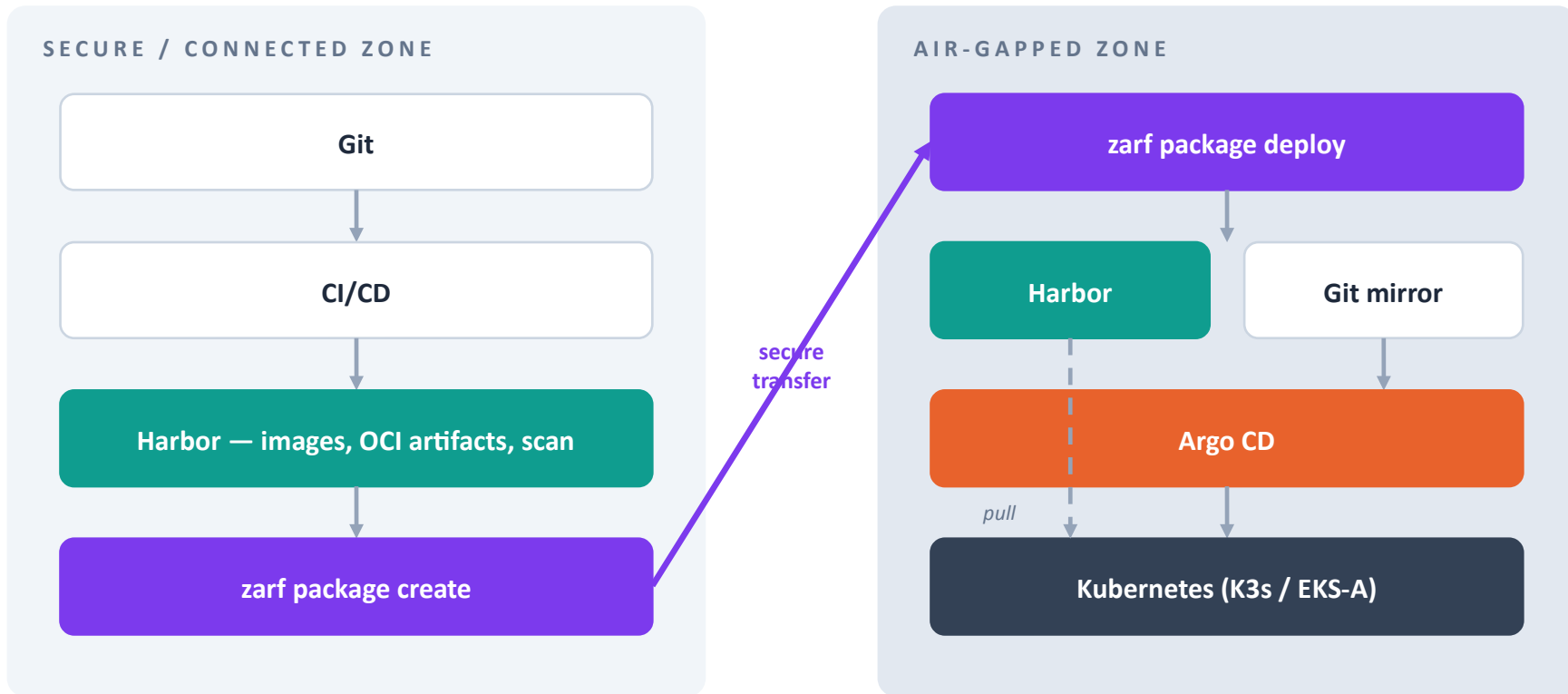
Argo CD

Keeps software IN its desired state

- Continuous day-2 operations
- Runs all the time, in the cluster
- Watches the internal Git mirror
- Detects and corrects drift

Common pattern: Zarf delivers Argo CD itself plus the Git repos it watches; Argo CD then owns day-2 operations.

Air-gapped reference architecture



Zarf is not "another Helm"



Helm

Kubernetes application → chart → install



Zarf

App + images + charts + manifests + dependencies
→ package → air-gapped site



Argo CD

Git → desired state → reconcile → Kubernetes



Harbor

Images, charts, OCI artifacts → store · scan ·
replicate

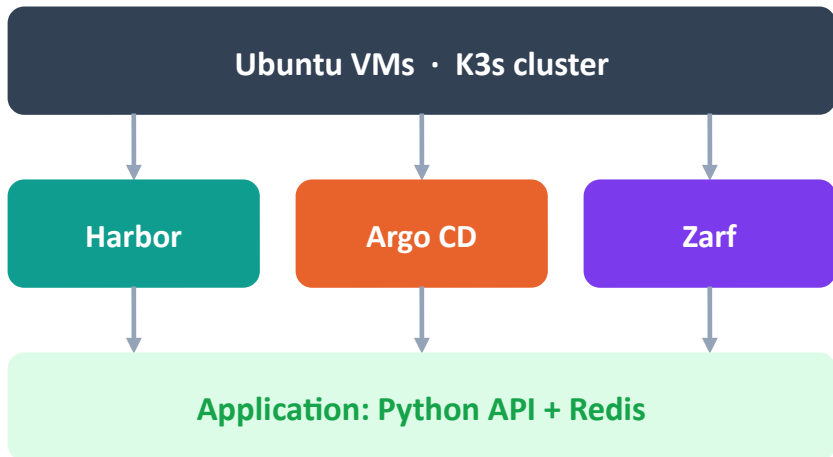
08

Hands-on Labs

Deploy a Python API + Redis application through all four tools —
then prove it works offline

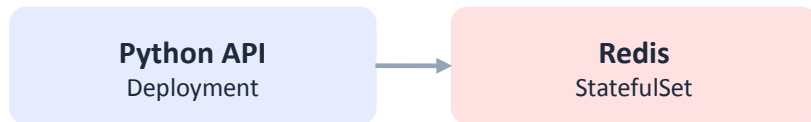


Lab architecture

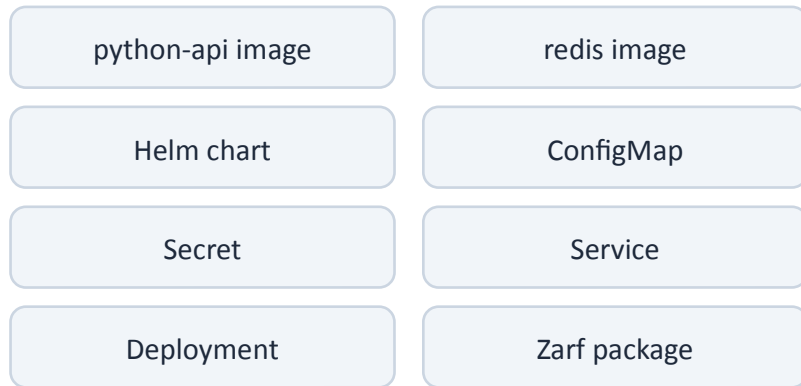


Prerequisites: K3s, kubectl, Helm 3, Docker or Podman, Zarf CLI, a Git server (Gitea), a DNS name and CA for Harbor.

The application

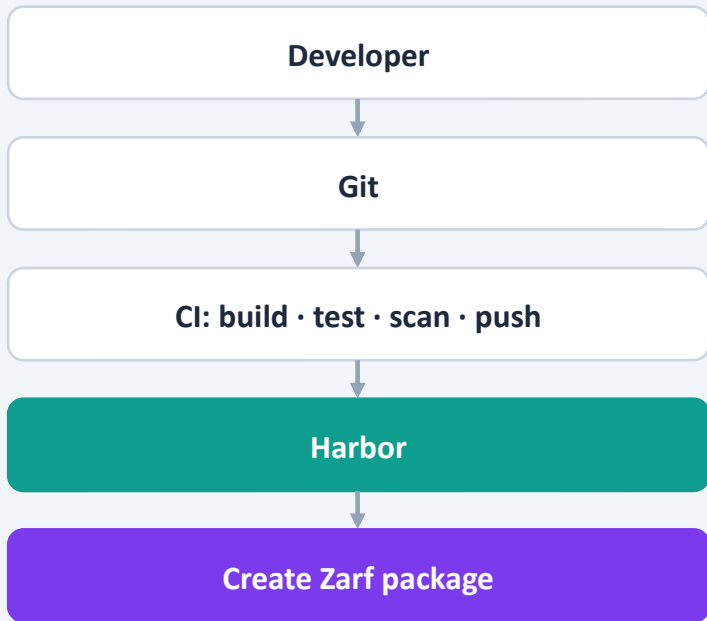


Artifacts produced

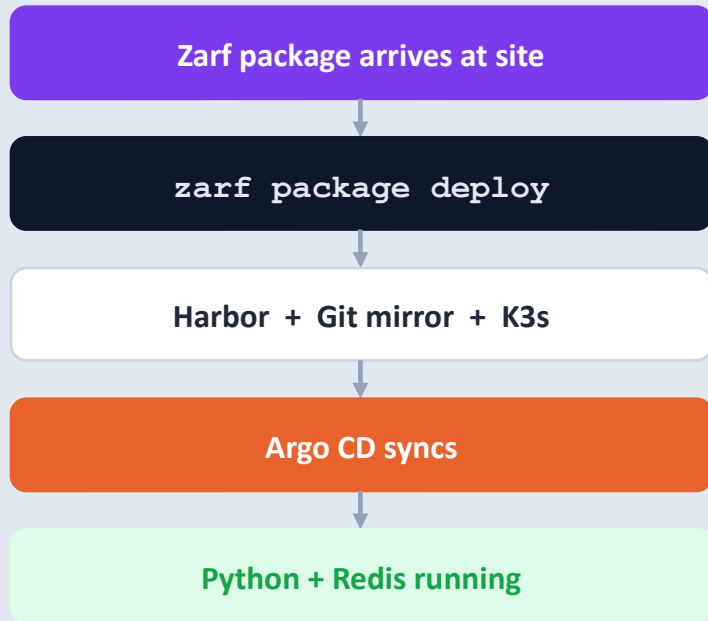


Example flow: build phase and deploy phase

CONNECTED BUILD PHASE



AIR-GAPPED DEPLOY PHASE



Lab 1: Create and release a Helm chart

Goal

Package the Python API + Redis as a chart and manage its release lifecycle.

Checkpoints

- Explore Chart.yaml, values.yaml, templates/
- Lint and render before installing
- Release visible in helm list
- Upgrade to 3 replicas, then roll back
- Explain why rollback created revision 3

```
helm create python-redis
# edit values.yaml: image, replicaCount, service
helm lint ./python-redis
helm template ./python-redis | less

helm install python-redis ./python-redis \
  -n demo --create-namespace
helm list -n demo
kubectl get pods,svc -n demo

helm upgrade python-redis ./python-redis \
  -n demo --set replicaCount=3
helm history python-redis -n demo
helm rollback python-redis 1 -n demo
```

Lab 2: Push and scan in Harbor

Goal

Store the image and chart in Harbor, scan them, and make K3s pull from Harbor.

Checkpoints

- Image visible under training/python-api
- Scan report reviewed; severities discussed
- Chart pushed as an OCI artifact
- Robot account created for K3s pulls
- registries.yaml configured on every node

```
docker login harbor.example.com
docker tag python-api:1.0 \
  harbor.example.com/training/python-api:1.0
docker push \
  harbor.example.com/training/python-api:1.0

# Harbor UI: training > python-api > 1.0 > Scan

helm package ./python-redis
helm registry login harbor.example.com
helm push python-redis-0.1.0.tgz \
  oci://harbor.example.com/training
```

Lab 3: GitOps with Argo CD

Goal

Let Argo CD deploy the chart from Git, then watch it sync changes and heal drift.

Checkpoints

- Argo CD UI reachable; admin login works
- Application shows Synced / Healthy
- Commit 2 → 3 replicas; observe sync
- Manual kubectl scale is reverted
- Disable selfHeal; observe OutOfSync

```
kubectl create namespace argocd
U=https://raw.githubusercontent.com/argoproj/argo-cd
kubectl apply -n argocd \
  -f $U/stable/manifests/install.yaml
kubectl -n argocd get secret \
  argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d

kubectl apply -f application.yaml
# values.yaml: replicaCount: 2 -> 3
git commit -am "scale to 3" && git push

# create drift
kubectl scale deploy python-redis \
  -n demo --replicas=1
```

Lab 4: Package and deploy with Zarf

Goal

Build a Zarf package online, then deploy it with the network disconnected.

Checkpoints

- Package created; name shows the architecture
- Package inspected, SBOMs reviewed
- Lab VM network disabled before deploy
- zarf init and deploy succeed offline
- Images listed in the internal registry

```
# connected machine
zarf tools download-init
zarf package create . --confirm
zarf package inspect \
  zarf-package-python-redis-amd64-1.0.0.tar.zst

# air-gapped cluster (network off)
zarf init --confirm
zarf package deploy \
  zarf-package-python-redis-amd64-1.0.0.tar.zst \
  --confirm
zarf package list
zarf tools registry catalog
helm list -A
```

Capstone: the end-to-end exercise

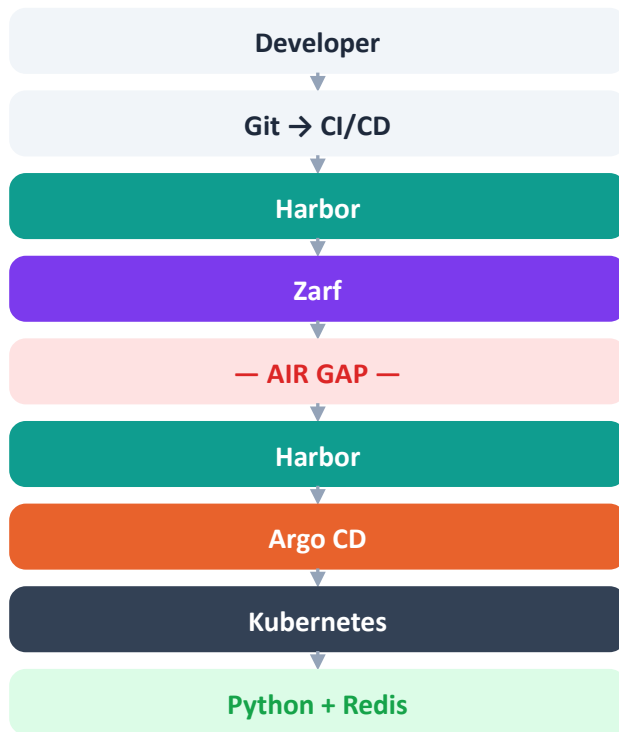
Scenario Your company runs Kubernetes in a secure data centre with no Internet connectivity. Deliver the Python + Redis application.

Requirements

- Private container registry
- Vulnerability scanning
- GitOps
- Helm-based deployment
- Air-gapped delivery

Deliverables

- Architecture diagram
- zarf.yaml
- Argo CD Application
- Release runbook
- 5-minute demo



Where could the deployment fail?

Pod: ImagePullBackOff

Investigate Harbor

- Image and tag exist?
- Pull credentials / robot account?
- Harbor CA trusted by containerd?
- Blocked by vulnerability policy?
- registries.yaml on this node?

```
kubectl describe pod <pod>  
k3s crictl pull <image>
```

Argo CD: OutOfSync

Investigate Git

- Wrong branch or path?
- Helm values file missing?
- Manual drift in cluster?
- Fields mutated by controllers?
- Repo reachable from Argo CD?

```
argocd app diff python-redis  
argocd app get python-redis
```

Zarf deploy fails

Investigate the package

- Image missing from zarf.yaml?
- Architecture mismatch?
- Cluster not initialised?
- Registry reachable / full?
- CRD needed before the chart?

```
zarf package inspect <pkg>  
zarf package deploy <pkg> \  
-l debug
```

REMEMBER THIS

One mental model



Helm packages → Harbor stores → Zarf transports → Argo CD reconciles → Kubernetes runs

Learning path: one delivery story, five phases

1

Packaging

Helm

How do I package
my Kubernetes
application?

2

Artifacts

Harbor

Where do I store
images and artifacts
securely?

3

GitOps

Argo CD

How do I
continuously
maintain desired
state?

4

Air-gap

Zarf

How do I get
everything into an
offline site?

5

Platform

All four

How do they
combine on K3s /
EKS Anywhere?

Teach the four tools as one software-delivery story, not as four independent products.

Questions?

Official documentation for further study



Helm

helm.sh/docs



Harbor

goharbor.io/docs



Argo CD

argo-cd.readthedocs.io



Zarf

docs.zarf.dev



K3s private registries

docs.k3s.io/installation/private-registry